

美股新闻分析 Agent —— 项目实施计划书 (Codex 执行版)

使用方法：把整份文档贴给 Codex（或在 VSCode 里让 Codex 读取这个文件），让它按第 11 节的任务清单 T1→T15 顺序实现。本文档已经把技术选型、数据结构、Prompt、配置文件内容都定好了，Codex 只需要照着写代码、跑通、自测即可，不需要重新做架构决策。

1. 项目目标与范围

核心需求（用户原始诉求）：

- 1. 7x24 监控美股相关新闻，尽量做到"新闻一出就抓到"。
- 2. 新闻抓到后立刻用 AI 判断：利好/利空、影响哪个板块/个股、大概什么时候会体现在股价上。
- 3. 项目里有一个 `watchlist.txt`（持仓 notepad），Agent 只针对和这些持仓**有关联**的新闻做详细分析；无关新闻直接跳过，不刷屏。
- 4. 每天美股开盘前，自动总结"昨晚收盘到现在"的新闻和影响，判断是否已经被市场 price in，预测今天哪些板块可能承压/受益。
- 5. 项目同步到 GitHub。

本阶段（MVP / Demo）范围：

- 纯后端 Python 项目，命令行运行（`python main.py`），**不做前端**。
- 在 VSCode 本地跑，单进程，用免费数据源 + OpenAI API。
- 只做"监控 + 分析 + 输出报告"，不做下单、不接券商账户。

明确不做事（non-goals，避免范围膨胀）：

- 不追求毫秒级真正实时推送（免费数据源做不到，下面会解释，用"高频轮询"代替）。
- 不做美股以外市场（港股/A股等）。
- 不做回测/历史准确率统计（放进第 12 节"后续扩展"）。

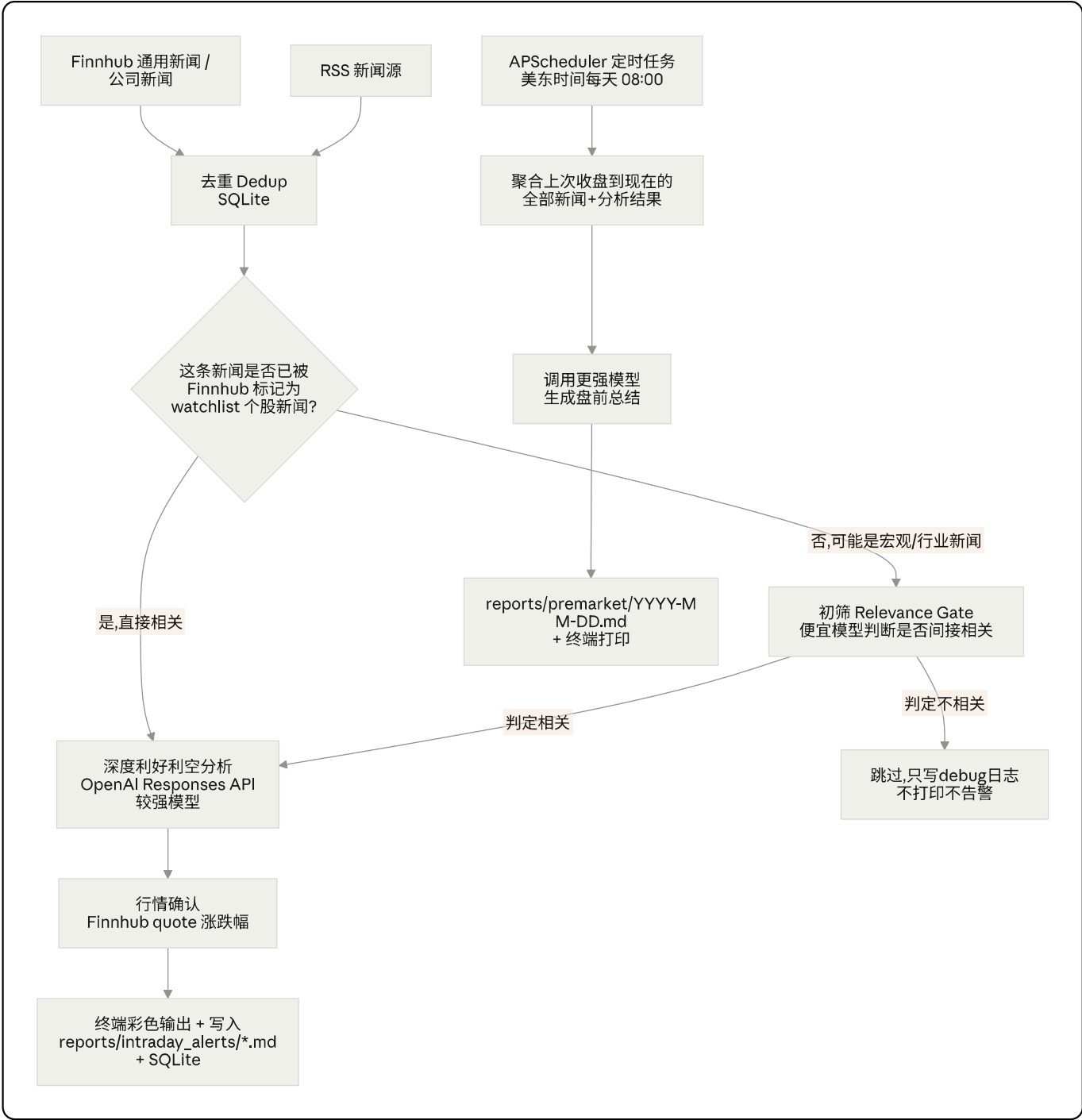
2. 技术选型（已根据你的回答确定）

类别	选择	说明
语言	Python 3.11+	
新闻源	Finnhub 免费版 <code>news</code> / <code>company-news</code> + RSS（CNBC / MarketWatch / Yahoo Finance / Nasdaq）	免费方案，见第 6.2/6.3 节

类别	选择	说明
行情/基本面数据	Finnhub 免费版 <code>(quote)</code> / <code>(stock/profile2)</code>	用于判断涨跌幅 + 行业归类
AI 分析引擎	OpenAI API (Responses API <code>(/v1/responses)</code>), 不要用旧的 <code>chat.completions</code> ——OpenAI 官方已经把 Responses API 定为新项目的默认推荐, 且 GPT-5.4 之后的推理模型在 Responses API 下工具调用支持更完整)	两级模型策略, 见第 6.5/6.6 节
任务调度	APScheduler	轮询任务 + 每日盘前定时任务
本地存储	SQLite (单文件, 无需装数据库服务)	新闻去重 + 分析结果历史
输出	终端彩色打印 (<code>(rich)</code>) + 本地 Markdown 报告 + 日志文件	不做前端
版本管理	Git + GitHub	见第 10 节

 OpenAI 的具体模型名 (GPT-5.4 / GPT-5.4-mini / GPT-5.4-nano 等) 更新很快, 写代码时把模型名做成环境变量 (见第 7 节 `(.env)`), **实现前去 platform.openai.com 的 dashboard 确认当前账号下可用的模型名**, 不要硬编码到代码逻辑里。

3. 系统架构



主循环（轮询，默认每 90 秒一轮）负责实时部分；APScheduler 的 cron 任务负责每日盘前总结。两者共享同一份 SQLite 历史数据。

4. 目录结构

```
us-stock-news-agent/
├─ README.md
├─ PROJECT_PLAN.md           # 本文档
├─ requirements.txt
├─ .env.example
├─ .gitignore
```

```
├─ config.yaml
├─ watchlist.txt          # 持仓 notepad, 用户手动编辑
├─ data/
│   ├─ news_history.sqlite3  # 去重+历史记录 (gitignore)
│   └─ ticker_meta.json     # watchlist 各ticker的行业/公司名缓存
├─ reports/
│   ├─ intraday_alerts/    # 每日实时告警markdown (gitignore, 量大)
│   └─ premarket/         # 每日盘前总结 (可选保留进git作为存档)
├─ logs/
│   └─ agent.log
├─ src/
│   ├─ __init__.py
│   ├─ config.py
│   ├─ models.py
│   ├─ watchlist.py
│   ├─ dedup_store.py
│   ├─ ticker_meta.py
│   ├─ news_sources/
│   │   ├─ __init__.py
│   │   ├─ finnhub_client.py
│   │   └─ rss_client.py
│   ├─ relevance_gate.py
│   ├─ analyzer.py
│   ├─ price_check.py
│   ├─ reporter.py
│   ├─ premarket_summary.py
│   └─ scheduler.py
├─ main.py
└─ scripts/
    └─ push_to_github.sh
```

5. 核心数据结构

设计原则：枚举值 (sentiment / confidence 等) 统一用**英文**，方便代码里写判断逻辑；给人看的解释文字用**中文**。展示层 (reporter.py) 负责把英文枚举映射成中文标签。

python

```

from dataclasses import dataclass
from datetime import datetime
from typing import Literal, Optional

@dataclass
class NewsItem:
    id: str # hash(url 或 headline+source+published_at)
    source: str # "finnhub" | "rss:cnbc" | "rss:marketwatch" | ...
    headline: str
    summary: str
    url: str
    published_at: datetime # 统一存UTC
    category: Literal["company", "general", "macro"]
    tagged_tickers: list[str] # Finnhub已经关联好的ticker (如果有)

@dataclass
class ImpactAnalysis:
    news_id: str
    ticker: str
    company_name: str
    sector: Optional[str]
    sentiment: Literal["bullish", "bearish", "neutral"]
    confidence: Literal["high", "medium", "low"]
    magnitude: Literal["large", "medium", "small"]
    time_horizon: Literal["intraday", "short_term_days", "medium_long_term"]
    priced_in: Literal["priced_in", "partially_priced_in", "not_priced_in", "unclear"]
    reasoning_zh: str # 1-3句中文解释
    price_change_pct_since_news: Optional[float] # 来自price_check.py, 可能为None
    created_at: datetime

```

6. 模块详细设计

6.1 watchlist.py —— 持仓 Notepad

watchlist.txt 格式 (纯文本, 一行一个ticker, (#)开头是注释) :

```

# 我的持仓 watchlist — 改完保存即可, 下一轮询周期自动生效, 不需要重启程序
NVDA
TSLA
AAPL
MSFT
AMD

```

要求:

- 每个轮询周期检查文件 mtime，变化了就重新加载，**不需要重启进程**。
- 解析时忽略空行和`#`注释，ticker统一转大写、去空格。

6.2 Finnhub 客户端 (`news_sources/finnhub_client.py`)

封装以下几个函数（具体字段以 Finnhub 官方文档 <https://finnhub.io/docs/api> 为准，实现时核对一遍）：

- `get_general_news()` → `GET /news?category=general&token=KEY`
- `get_company_news(symbol, from_date, to_date)` → `GET /company-news?symbol=AAPL&from=2026-06-16&to=2026-06-17&token=KEY`
- `get_quote(symbol)` → `GET /quote?symbol=AAPL&token=KEY`（返回当前价/涨跌幅/前收盘）
- `get_profile(symbol)` → `GET /stock/profile2?symbol=AAPL&token=KEY`（拿行业 `finnhubIndustry`、公司名）

注意点：

- 免费版限速 60 次/分钟，要做简单限速（比如固定间隔 sleep，或令牌桶），遇到 429 做一次退避重试。
- `get_profile` 不需要每个轮询周期都调，**启动时为 watchlist 里每个 ticker 各调一次，缓存进 `data/ticker_meta.json`，之后每天刷新一次即可。**

6.3 RSS 客户端 (`news_sources/rss_client.py`)

用 `feedparser` 抓取，候选信源（实现时务必逐个验证 URL 仍然有效，单个失效不能影响整体）：

名称	URL (候选，需验证)
CNBC Top News	<code>https://search.cnn.com/rs/search/combinedcms/view.xml?partnerId=wrss01&id=100003114</code>
MarketWatch Top Stories	<code>http://feeds.marketwatch.com/marketwatch/topstories/</code>
Yahoo Finance	<code>https://finance.yahoo.com/news/rssindex</code>
Nasdaq Markets	<code>https://www.nasdaq.com/feed/rssoutbound?category=Markets</code>

要求：每个 feed 单独 try/except，失败只记日志、不抛异常；统一转换成 `NewsItem` (`category` 先标 `"general"` 或 `"macro"`，没有 `tagged_tickers`)。

6.4 `dedup_store.py`

SQLite 表 `seen_news(id TEXT PRIMARY KEY, first_seen_at TEXT, headline TEXT)`。每轮先查

id 是否存在，不存在才进入下游处理并插入。建议加一个定时清理（保留最近14天），避免库无限增长。

6.5 `relevance_gate.py` —— 初筛（只用于没被 Finnhub 直接打标的新闻）

逻辑：

1. 如果 `news.tagged_tickers` 里有任何 ticker 命中 watchlist → **直接判定相关**，跳过初筛，进入深度分析（6.6）。
2. 否则（通用新闻/宏观新闻），调用便宜模型做一次相关性判断：把 watchlist 的 ticker+行业列表喂给模型，问它"这条新闻会不会通过行业、供应链、竞争对手、宏观利率/汇率/大宗商品等关系，间接影响列表里的任意标的"，返回：

```
json
{"relevant": true, "candidate_tickers": ["NVDA", "AMD"], "reason_brief": "中国出口管制消
```

3. `relevant=false` → 整条跳过，只写 debug 日志，**不进入终端正式输出**（这是用户要的"无关就直接跳过"）。

6.6 `analyzer.py` —— 深度利好利空分析

对每个 candidate ticker 单独调用一次"正式分析"，用 OpenAI Responses API，配合 Structured Outputs (`text.format` 指定 `json_schema`) 强制输出符合第5节 `ImpactAnalysis` 结构的 JSON。

System Prompt（草稿，可直接用）：

你是一名资深美股新闻分析师。只根据给定的新闻内容和公司/行业背景，做客观、简洁的利好/利空（bullish/bearish/neutral）判断。

不输出任何买卖建议，不做价格预测，只分析新闻本身对该标的可能造成的影响方向、强度、时间维度，以及这个影响是否已经被市场 price in。

所有解释性文字必须用中文。严格按照给定的 JSON schema 输出，不要输出 schema 之外的任何文字或 markdown 标记。

User Prompt 模板：

新闻标题：{headline}
新闻摘要：{summary}
来源：{source}
发布时间（UTC）：{published_at}
需要分析的标的：{ticker}（{company_name}，所属行业：{sector}）
该标的当前价格信息（可能为空）：{price_info}

请分析这条新闻对该标的可能的影响。

JSON Schema (结构对应第5节 ImpactAnalysis, 去掉不需要模型生成的字段如 news_id/created_at) :

```
json
{
  "name": "impact_analysis",
  "schema": {
    "type": "object",
    "properties": {
      "sentiment": {"type": "string", "enum": ["bullish", "bearish", "neutral"]},
      "confidence": {"type": "string", "enum": ["high", "medium", "low"]},
      "magnitude": {"type": "string", "enum": ["large", "medium", "small"]},
      "time_horizon": {"type": "string", "enum": ["intraday", "short_term_days", "medium_term_days", "long_term_years"]},
      "priced_in": {"type": "string", "enum": ["priced_in", "partially_priced_in", "not_priced_in"]},
      "reasoning_zh": {"type": "string"}
    },
    "required": ["sentiment", "confidence", "magnitude", "time_horizon", "priced_in", "reasoning_zh"],
    "additionalProperties": false
  },
  "strict": true
}
```

容错要求: 调用失败/解析失败时重试一次, 仍失败则记录错误、跳过这条, **不能让主循环崩溃**。

6.7 price_check.py

拿 `get_quote(ticker)` 算出 $(c - pc) / pc$ 作为"自上一次收盘以来涨跌幅", 作为 `priced_in` 判断的辅助参考数值 (不是唯一依据, 定性判断仍由LLM给出)。Finnhub 免费版对盘前/盘后数据可能有限制, **如果取不到就明确标注"无盘前数据, 仅供参考", 不要伪造数值**。

6.8 reporter.py

- 终端输出: 用 `rich` 的 Panel/Table, bullish 用绿色、bearish 用红色、neutral 用黄色, 一条新闻一张卡片, 包含 ticker、中文结论、reasoning、priced_in 状态、新闻链接。
- 同时把同样内容 append 写入 `reports/intraday_alerts/YYYY-MM-DD.md`。

6.9 premarket_summary.py

聚合"上一次美股收盘 (美东16:00) 到现在"区间内的全部 `NewsItem` + `ImpactAnalysis`, 按 ticker/sector 分组后, 调用一次更强模型 (一天只跑一次, 可以用更贵但更强的模型) 生成盘前简报, 至少包含:

1. 隔夜整体市场情绪概览 (美股期货方向、亚太/欧洲市场联动、美元/利率/大宗商品动态)
2. 与你持仓直接相关的新闻清单 + 逐条判断
3. 今天开盘前可能承压 / 受益的板块预测

4. 哪些消息看起来已经被市场 price in (结合 price_check 的隔夜涨跌幅数据), 哪些还没充分反映

输出保存到 `reports/premarket/YYYY-MM-DD.md` 并打印到终端。提供一个 `--run-now` 的 CLI 参数, 方便随时手动触发测试, 不必真等到第二天早上。

6.10 `scheduler.py` / `main.py`

用 APScheduler 配置两个任务:

- **轮询任务** (`IntervalTrigger`, 默认每 90 秒): 执行"抓新闻 → 去重 → 初筛 → 深度分析 → 行情确认 → 输出"全流程。
- **每日盘前任务** (`CronTrigger(timezone="America/New_York", hour=8, minute=0)`): 用 `zoneinfo`/`pytz` 的"美东时区"配置, APScheduler 会自动处理夏令时切换, 不需要手动算 UTC 偏移。

要求:

- 每个 pipeline 步骤都要 try/except, 单条新闻或单个数据源失败只记日志, 绝不能让整个 agent 退出。
- 支持 `Ctrl+C` 优雅退出, 确保 SQLite 连接正常关闭。

7. 配置文件内容

`.env.example`

```
FINNHUB_API_KEY=your_finnhub_key
OPENAI_API_KEY=your_openai_key

# 模型名请在实现前去 platform.openai.com dashboard 核实当前账号可用的最新型号
OPENAI_MODEL_GATE=gpt-5.4-nano           # 初筛用, 要便宜快
OPENAI_MODEL_ANALYZE=gpt-5.4-mini        # 单条深度分析用
OPENAI_MODEL_SUMMARY=gpt-5.4             # 每日盘前总结用, 一天一次可以用更强模型

POLL_INTERVAL_SECONDS=90
PREMARKET_SUMMARY_HOUR=8
PREMARKET_SUMMARY_TIMEZONE=America/New_York
```

`config.yaml`

yaml

```
watchlist_file: watchlist.txt
output_language: zh
report_dir: reports
log_dir: logs

rss_feeds:
  - name: cnbc_top_news
    url: https://search.cnbc.com/rs/search/combinedcms/view.xml?partnerId=wrss01&id=100
  - name: marketwatch_top
    url: http://feeds.marketwatch.com/marketwatch/topstories/
  - name: yahoo_finance
    url: https://finance.yahoo.com/news/rssindex
  - name: nasdaq_markets
    url: https://www.nasdaq.com/feed/rssoutbound?category=Markets
```

requirements.txt

```
openai>=1.50
requests
feedparser
apscheduler
rich
python-dotenv
pyyaml
tzdata
```

8. 验收标准 (demo 跑起来后应该满足)

1. `python main.py` 启动后，终端打印类似"Agent已启动，正在监控 N 个watchlist标的 + M 个RSS源"。
2. 轮询过程中，一旦有新新闻命中 watchlist（直接或行业间接相关），终端立刻打印一张带颜色的利好/利空分析卡片，并同步写入 `reports/intraday_alerts/当天日期.md`。
3. 无关新闻能在日志里看到"已跳过"记录，但**不会**出现在终端正式告警区域（避免刷屏噪音）。
4. 修改 `watchlist.txt` 后不需要重启程序，下一个轮询周期自动生效。
5. 跑 `python -m src.premarket_summary --run-now` 能立即生成一份完整的中文盘前总结 Markdown，不需要等到真正的时间点。
6. 模拟网络超时/API报错（比如断网测试），程序不整体崩溃，日志里有清晰错误记录，下一轮继续正常工作。
7. `.env` 里的 key 不会被打印到日志，也不会出现在 git 历史里。

9. GitHub 同步方案

`.gitignore`

```
.env
__pycache__/*
*.pyc
.venv/
data/*.sqlite3
data/ticker_meta.json
logs/
reports/intraday_alerts/
```

`reports/premarket/` 建议**保留**进 git，作为每日盘前总结的历史存档，方便以后回顾对照实际走势；`intraday_alerts/` 量太大，建议忽略。

操作步骤

1. `git init`
2. 按上面内容写 `.gitignore`
3. `git add . && git commit -m "init: us stock news agent skeleton"`
4. 创建远程仓库（二选一）：
 - 网页上手动新建一个空仓库（**不要**勾选自动生成README，避免和本地冲突）
 - 或用 GitHub CLI：先 `gh auth login`，再 `gh repo create us-stock-news-agent -private --source=. --remote=origin`
5. `git remote add origin git@github.com:<你的用户名>/us-stock-news-agent.git`（如果上一步用 `gh repo create` 已自动配好可跳过）
6. `git branch -M main`
7. `git push -u origin main`

日常同步建议

不建议每条新闻都自动 commit（会让历史很乱、也没必要）。推荐：代码改动手动 `git add . && git commit -m "..."` `&& git push`；每日盘前报告可选用下面脚本手动或加进 cron 定时归档：

```
bash
```

```
#!/usr/bin/env bash
# scripts/push_to_github.sh
set -e
cd "$(dirname "$0")/.."
git add reports/premarket/
git commit -m "chore: archive premarket report $(date +%Y-%m-%d)" || echo "无变更可提交"
git push origin main
```

10. 给 Codex 的任务清单（按顺序执行，每步完成后做一次最小自测）

步 任务 骤

- T1 按第4节创建目录结构与空文件，写好 `requirements.txt` / `.env.example` / `.gitignore`，`pip install -r requirements.txt`
- T2 实现 `config.py` + `watchlist.py`：读取 `.env` / `config.yaml`，解析 `watchlist.txt`，写个小脚本打印结果验证
- T3 实现 `finnhub_client.py`：四个函数 + 429重试/限速。用真实watchlist里一个ticker跑一次验证能拿到数据
- T4 实现 `rss_client.py`：遍历config里的feed抓取并转成 `NewsItem`，验证单个feed失效不会让整体崩溃
- T5 实现 `models.py` + `dedup_store.py`：dataclass定义 + SQLite表初始化/查重/写入
- T6 实现 ticker 的 profile/sector 缓存：启动时为watchlist每个ticker调一次profile2，存进 `data/ticker_meta.json`
- T7 实现 `relevance_gate.py`：先用mock数据跑通分支逻辑，再接入真实OpenAI调用
- T8 实现 `analyzer.py`：按6.6节Prompt+JSON Schema实现，确保Structured Outputs解析稳定，失败有重试+兜底
- T9 实现 `price_check.py`：算涨跌幅，处理拿不到数据的情况
- T10 实现 `reporter.py`：先用假数据测试终端样式和markdown格式，再接入真实pipeline
- T11 实现 `premarket_summary.py`：聚合逻辑 + 独立summary prompt，做好 `--run-now` 参数
- T12 实现 `scheduler.py` + `main.py`：串联全部模块，配置两个APScheduler任务，加graceful shutdown
- T13 端到端测试：完整跑 `python main.py`，逐条核对第8节验收标准

步骤任务

T14 按第9节完成 git 初始化 + 创建GitHub远程仓库 + 首次push

T15 (可选) 写一份简明 `README.md`，说明如何配置 `.env`、如何运行、如何编辑 `watchlist.txt`

11. 后续可扩展方向（明确不在本次 MVP 范围内）

- 升级到付费数据源（Polygon / Benzinga）实现真正的 push 式实时新闻推送，替代轮询。
 - 加 Telegram / Slack / Email 即时通知通道，替代或补充终端输出。
 - 做一个简单的 Web Dashboard 做可视化（等demo稳定后再考虑）。
 - 历史准确率回测：记录每次判断后 N 小时/N 天该ticker实际涨跌，统计模型判断准确率，反哺Prompt迭代。
 - 美股交易日历/假期感知，避免休市日触发无意义的盘前总结。
 - 接入期权异动、资金流、社交媒体舆情等另类数据源。
-

12. 重要声明

本工具仅作个人新闻聚合与辅助分析用途。所有"利好/利空"判断均由大语言模型基于公开新闻文本生成，**不构成投资建议**，可能存在错误、滞后或误判，请勿作为唯一决策依据进行实盘交易。请妥善保管 `.env` 中的 API Key，切勿提交到任何公开仓库。